Dual-Mode Gesture and Smartphone Application-Controlled Robotic Vehicle: Design, Implementation, and Future Autonomous Expansion

Huynh Nhan Thien Tri Tin (SE201630)[1], Tran Dinh Khoi Nguyen (SE200558)[2],
Nguyen Quang Huy Dat (SE201415)[3], and Le Tan Loc (SE201412)[4]

Group 2, IC Design Program, Class IC2010
FPT University
Ho Chi Minh City, Vietnam

***Abstract*—The primary objective of this project is to design, construct, and program a versatile robotic vehicle powered by Arduino Nano microcontrollers. The system features a highly interactive dual-mode control mechanism, shifting away from conventional tactile remotes. The first mode utilizes a wearable controller equipped with a GY-61 analog accelerometer, allowing the user to steer the robot intuitively through physical hand gestures by measuring spatial tilt angles. The second mode utilizes a smartphone application via wireless Bluetooth technology, providing a reliable graphical interface. While the vehicle is highly versatile and cross-compatible with generic third-party remote apps on the market, a custom-built Android application was developed to provide exclusive access to advanced features, including a future autonomous driving mode. Experimental results demonstrate low latency (under 100ms), high mechanical stability, and clear, actionable roadmaps for resolving current mechanical drift and integrating ultrasonic autonomous navigation.**

***Index Terms*—Gesture Control, Arduino Nano, Bluetooth HC-05, L298N Motor Driver, GY-61 ADXL335, Human-Machine Interface (HMI), Mobile Application.**

I. Introduction

In the rapidly evolving field of robotics and embedded systems, intuitive Human-Machine Interfaces (HMI) are becoming increasingly vital. Traditional joystick controllers and wired push-button remote systems, while functionally effective, often lack the natural intuitiveness required for seamless human-robot interaction in modern applications. This project aims to address these interactive limitations by developing a dual-mode controlled robotic vehicle. The proposed system integrates an analog 3-axis accelerometer (GY-61) into a wearable glove, enabling gesture-based control where the physical kinematics of the user's hand directly dictate the vehicle's locomotion.

Recognizing the ubiquity of modern smartphones, a secondary control mode via Bluetooth is introduced. A major design advantage of this robotic vehicle is its broad compatibility. By utilizing the standard Universal Asynchronous Receiver-Transmitter (UART) serial communication protocol over a classic Bluetooth profile, the vehicle can be seamlessly controlled using various third-party generic Bluetooth joystick applications available on the Google Play Store, thereby maximizing its versatility. However, to fully unlock the system's potential, a custom-developed Android application is introduced. This proprietary app distinguishes itself by offering extensive UI customization, modern Android permission handling, and acting as the exclusive gateway to toggle the vehicle's future autonomous navigation mode.

During the initial prototyping phase, NRF24L01 radio transceiver modules utilizing the SPI protocol over the 2.4GHz band were extensively tested. However, these modules proved to be highly susceptible to environmental electromagnetic noise and power supply fluctuations, resulting in unstable connections, packet loss, and frequent disconnections. Consequently, the communication architecture was systematically transitioned to utilize paired HC-05 Bluetooth modules, establishing a significantly more reliable, robust, and easy-to-debug connection.

II. System Architecture

The overall system architecture is designed with modularity, low power consumption, and scalability in mind. It is divided into two primary interconnected hardware subsystems operating in a Master-Slave topology.

A. Gesture Transmitter Subsystem (Hand Controller)

This wearable unit acts as the primary sensory input and the "Master" node in the communication network. It continuously samples analog spatial data regarding the user's hand orientation, digitizes it, categorizes it into directional states, and transmits discrete character commands wirelessly. The conceptual block diagram of this subsystem, highlighting the data flow from the GY-61 sensor to the Arduino Nano and out through the HC-05 module, is illustrated in Fig. 1.

B. Receiver and Actuation Subsystem (Robotic Vehicle)

The core vehicle acts as the "Slave" execution unit. It remains in a listening state until a command is received via Bluetooth. Once validated, it translates these low-power Bluetooth commands into high-current logic signals for motor actuation. The system architecture of the vehicle, integrating the L298N motor driver, the central Arduino Nano, and the DC motors, is depicted in Fig. 2.

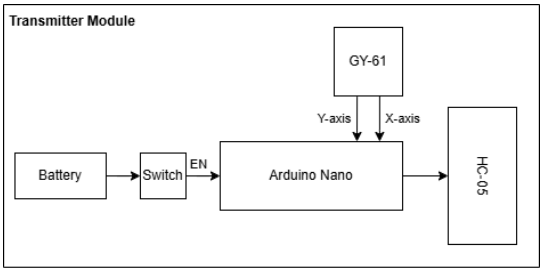

Fig. 1. Block diagram of the gesture-based transmitter (hand controller).

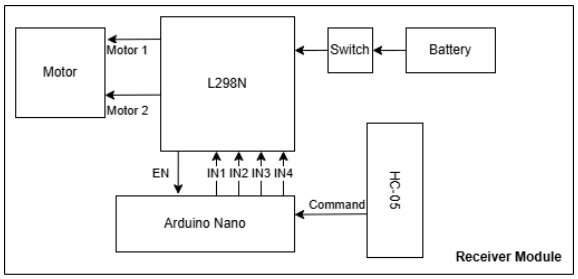

Fig. 2. Block diagram of the robotic vehicle receiver and actuation system.

III. Hardware Design and Implementation

A. Transmitter Wiring and Assembly

The GY-61 module (housing the ADXL335 chip) is an analog accelerometer that measures static acceleration due to gravity. As the user's hand tilts, the projection of gravity on the sensor's internal axes changes, altering the output voltage. The analog voltages corresponding to the X (pitch) and Y (roll) axes are connected directly to the ADC (Analog-to-Digital Converter) pins A0 and A1 of the Arduino Nano.

Power is supplied by high-capacity 18650 lithium-ion batteries routed through a physical switch for safety. The HC-05 Bluetooth module is wired to the Arduino's software serial pins (RX, TX) with appropriate voltage division logic. The precise schematic wiring is shown in Fig. 3, while the final physical assembly of the wearable glove is presented in Fig. 4.

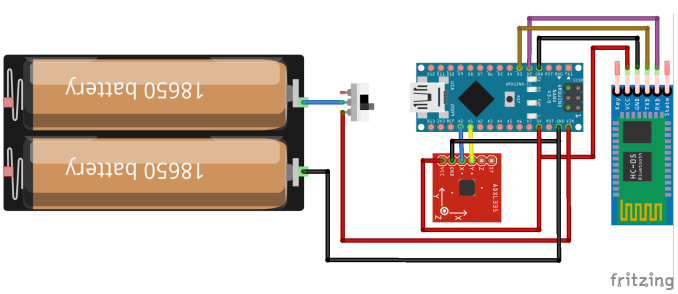

Fig. 3. Schematic wiring diagram of the wearable gesture controller.

B. Vehicle Chassis Wiring and Assembly

The robotic chassis is driven by a 4-Wheel-Drive (4WD) DC motor setup, controlled by an L298N dual H-bridge motor driver. The L298N serves as the critical intermediary between the low-power 5V logic signals of the Arduino Nano and the high-current demands of the geared DC motors.

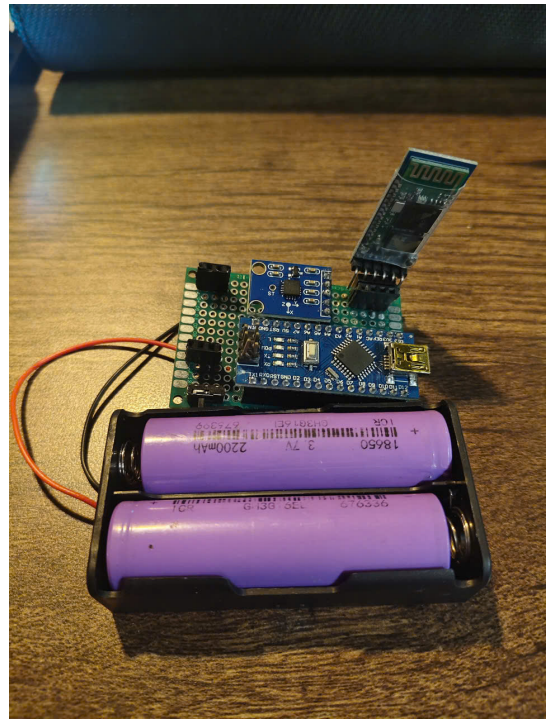

Fig. 4. Physical assembly of the wearable gesture controller (Transmitter).

As illustrated in the schematic (Fig. 5), the 18650 battery pack provides the main heavy-duty power directly to the L298N's 12V input. The driver's onboard 5V regulator is then utilized to power the Arduino Nano and the HC-05 slave module, eliminating the need for a secondary power source. Logic pins IN1 through IN4 dictate the rotational direction of the left and right motor banks. The completed physical vehicle chassis is shown in Fig. 6.

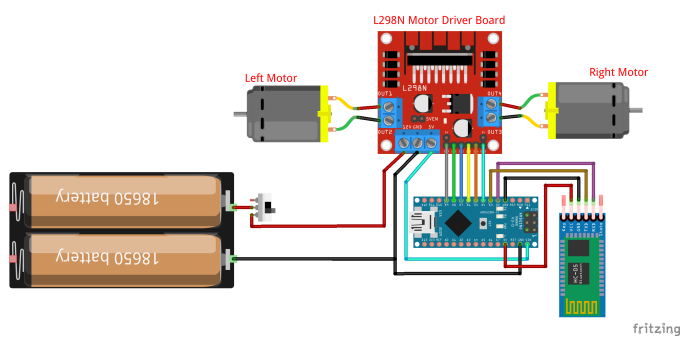

Fig. 5. Schematic wiring diagram of the robotic vehicle chassis.

IV. Software and Algorithm Design

A. Transmitter Algorithm (Gesture Recognition)

The embedded software on the transmitter utilizes a continuous polling loop to read the GY-61 sensor. A critical

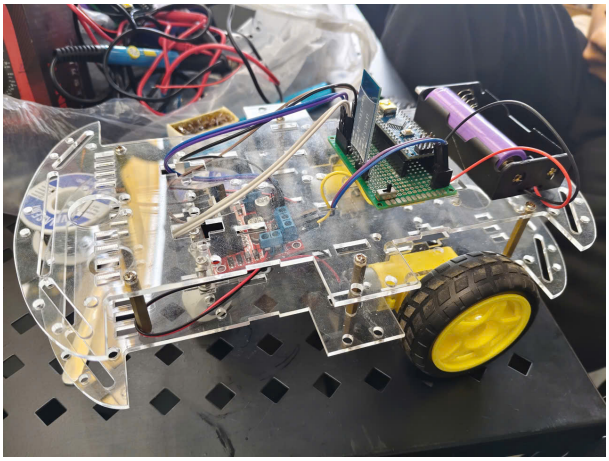

Fig. 6. Physical implementation of the robotic vehicle chassis (Receiver).

component of the software design is the implementation of a ”deadzone” thresholding algorithm (values between 280 and 380). This acts as a software filter to eliminate minor physiological tremors (hand shaking) and defines a clear neutral resting state (‘S’ - Stop).

The logical execution flow is visualized in Fig. 7. Upon initialization and Bluetooth pairing, the system enters a continuous read cycle. It sequentially evaluates the X_{val} and Y_{val} integers. If the tilt exceeds the predefined threshold bounds, it maps the gesture to a specific ASCII character (‘F’ for Forward, ‘B’ for Backward, ‘L’ for Left, ‘R’ for Right) and transmits it over the UART serial line.

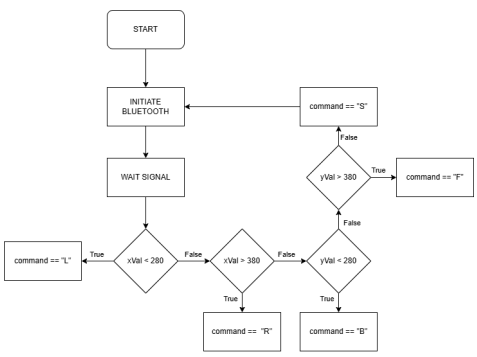

Fig. 7. Software flowchart detailing the hand controller’s gesture recognition logic.

TABLE I
ACCELEROMETER THRESHOLD MAPPING AND COMMAND LOGIC

Analog ADC Value	Gesture / Orientation	Transmitted Char
$X < 280$	Hand Tilted Forward	‘F’
$X > 380$	Hand Tilted Backward	‘B’
$Y < 280$	Hand Tilted Left	‘L’
$Y > 380$	Hand Tilted Right	‘R’
$280 \leq X, Y \leq 380$	Neutral / Flat Position	‘S’ (Stop)

B. Receiver Algorithm (Command Execution)

The receiver Arduino implements a highly efficient, non-blocking serial buffer check algorithm, detailed in Fig. 8. The microcontroller loops rapidly in a ”Wait State”. The condition `Serial.available()` acts as a trigger.

Once a data packet arrives, it extracts the valid ASCII character and feeds it into a `switch-case` state machine. For instance, receiving an ‘F’ command triggers the function to set the IN1/IN3 pins HIGH and IN2/IN4 pins LOW, driving all motors forward. If the signal is lost or an ‘S’ is received, the system defaults to an electronic brake state (all IN pins LOW).

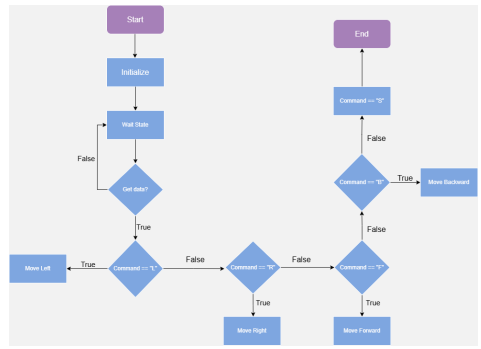

Fig. 8. Software flowchart of the robotic vehicle’s command execution loop.

C. Mobile Application Cross-Compatibility and Optimization

Because the vehicle’s embedded software is programmed to recognize standard ASCII characters over a generic SPP (Serial Port Profile) Bluetooth link, it possesses cross-compatibility with numerous third-party applications.

However, the custom-developed Android application (shown in Fig. 9) provides critical strategic advantages. It is specifically engineered with dedicated programming blocks to dynamically bypass Android 12+ strict Bluetooth permissions (preventing legacy Storage Request errors) and serves as a comprehensive control hub that generic applications cannot provide.

Fig. 9. Graphical User Interface (GUI) of the custom Android application.

V. EXPERIMENTAL RESULTS AND FUTURE WORK

A. Experimental Results and Limitations

Extensive field testing confirmed that the threshold calibration for the hand controller was highly effective. The robotic

vehicle responded instantly to hand tilts, boasting an estimated communication latency of less than 100 milliseconds. The custom smartphone application paired reliably with the HC-05 module without triggering OS-level crashes.

Despite the robust software performance, a mechanical shortcoming was observed. Due to the loose manufacturing tolerance and structural play in the generic yellow wheel-to-motor-shaft assemblies, the vehicle exhibits a noticeable tendency to drift slightly to the right when commanded to travel in a straight line. This mechanical misalignment will be addressed in future structural revisions utilizing tighter tolerances or closed-loop encoder feedback.

B. Future Autonomous Navigation

Although the current project focuses primarily on manual HMI control, the custom application and underlying architecture inherently support automated upgrades. Future versions will integrate the HC-SR04 ultrasonic sensor. The proposed application interface with automatic mode switching functionality is shown in Figure 11. If an obstacle is detected within a 15 cm radius, the microcontroller will generate a priority interrupt to stop the DC motors. Then, a micro-servo motor will scan the sensor to map the spatial clearance, allowing the vehicle's control system to turn along the clearest path—an advanced mode that can only be activated through the proprietary application.

Currently, the app and code loaded onto the vehicle are ready for the autopilot mode. However, due to time and budget constraints, the autopilot mode is not yet fully optimized and has not undergone testing. Therefore, it will be considered part of the project's future development.

Fig. 10. Proposed GUI layout featuring the autonomous navigation toggle.

Fig. 11. Toggle feature in control mode.

VI. Conclusion

This project successfully validated a fully functional, dual-controlled robotic vehicle. By meticulously integrating hardware wiring, defining rigorous sensor data processing algorithms with noise-filtering thresholds, and establishing robust Bluetooth communication, the project delivered a highly responsive HMI. The system's compatibility with generic market applications highlights its versatility, while the custom app ensures scalability. Acknowledging the current mechanical drifting issues provides a transparent and clear engineering roadmap for transitioning this manual platform into an advanced, structurally refined autonomous system.

Author Contributions

The successful completion of this project was the result of equal collaborative efforts from all members of Group 2. Each member contributed exactly **25%** to the overall workload, encompassing conceptualization, hardware assembly, embedded software development, mobile application design, and technical documentation. The specific distributions are outlined below:

- **Huynh Nhan Thien Tri Tin (SE201630):** (25%) Successful design and implementation of a dual-mode robotic vehicle utilizing a GY-61 accelerometer for intuitive gesture control.
- **Tran Dinh Khoi Nguyen (SE200558):** (25%) Development of a custom Android application equipped with dynamic permission handling algorithms to bypass generic connectivity issues on modern OS versions.
- **Nguyen Quang Huy Dat (SE201415):** (25%) Development of the embedded command execution algorithms, integration of the L298N motor actuation logic, and comprehensive field testing to evaluate system stability.
- **Le Tan Loc (SE201412):** (25%) Establishment of a reliable Master-Slave Bluetooth communication protocol, laying a structured foundation for future autonomous navigation upgrades.

References

[1] Arduino, "Arduino Nano Hardware Reference," *Arduino Official Documentation*, 2023.
[2] Analog Devices, "ADXL335: Small, Low Power, 3-Axis ±3 g Accelerometer Data Sheet," Rev. C, 2010.
[3] STMicroelectronics, "L298 Dual Full-Bridge Driver Datasheet," 2000.
[4] Guangzhou HC Information Technology, "HC-05 Bluetooth to Serial Port Module User's Manual," 2011.
[5] MIT Media Lab, "MIT App Inventor / Kodular Documentation," 2024. [Online]. Available: https://appinventor.mit.edu/
[6] Google, "Bluetooth permissions on Android 12 or higher," *Android Developers Documentation*, 2024.
[7] SparkFun Electronics, "Ultrasonic Ranging Module HC-SR04 Datasheet," 2014.
[8] J. Borenstein and Y. Koren, "Real-time obstacle avoidance for fast mobile robots," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 19, no. 5, pp. 1179-1187, Sept.-Oct. 1989.

APPENDIX

The development of this robotic vehicle involved extensive hands-on fabrication to ensure the structural and electrical integrity of the final product. This included prototyping control logic on breadboards, carefully soldering electronic components to establish stable connections resistant to vibration, and routing jumper wires methodically across the vehicle's chassis. The following images document the practical assembly phases of the project.

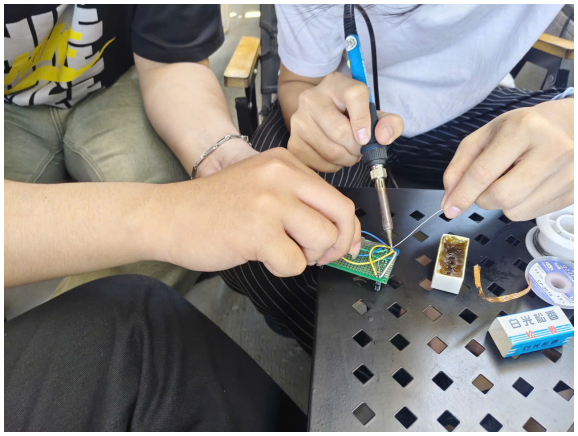

Fig. 12. Soldering and securing the electronic components to ensure stable connections.

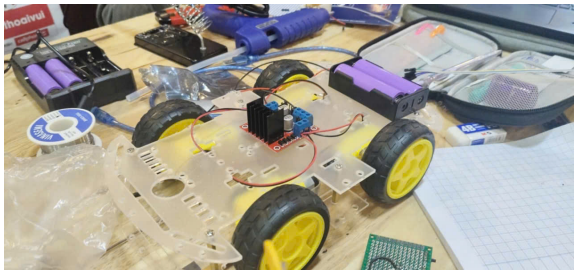

Fig. 13. Initial wiring diagram for a remote-controlled car.

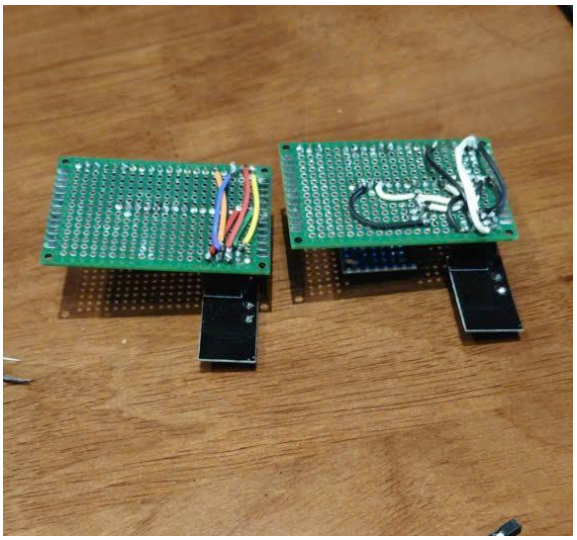

Fig. 14. The original controller used the GY-61.

Comment: Initial communication testing proved unstable, leading to the replacement with HC-05 Bluetooth modules for reliable transmission.

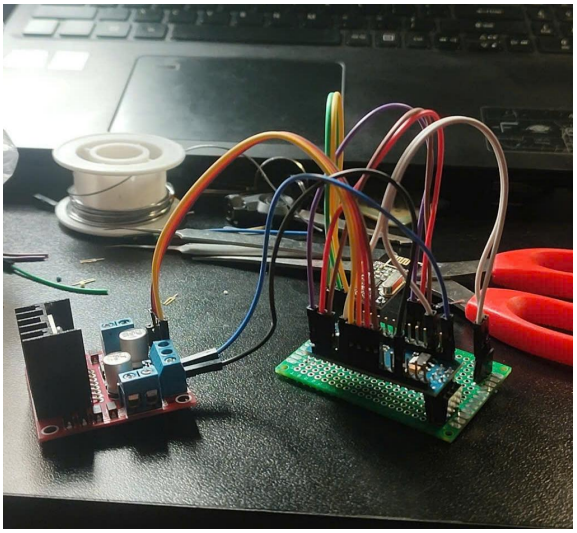

Fig. 15. Try different wiring methods to optimize space and test Bluetooth signal reception/transmission capabilities.

Comment: This phase involved researching the most optimal and neatest wiring methods, while simultaneously establishing the Master-Slave protocol for the two paired HC-05 modules.

To complement the physical gesture controller, a custom Android application was developed to provide a reliable graphical interface. The following screenshots illustrate the primary control layouts, connection handling, and advanced features of the application.

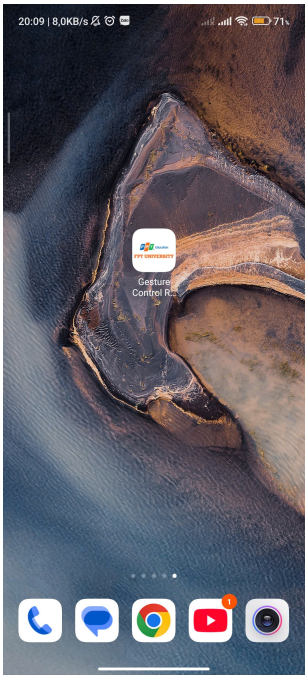

Fig. 16. External app interface

Fig. 17. App loading screen

Fig. 18. Pre-permission interface required

Fig. 19. The interface after granting permissions.